

**UNIVERSIDADE REGIONAL INTEGRADA DO ALTO URUGUAI E DAS
MISSÕES CÂMPUS URI ERECHIM
DEPARTAMENTO DE ENGENHARIAS E CIÊNCIA DA COMPUTAÇÃO
CURSO DE CIÊNCIAS DA COMPUTAÇÃO**

**ALISSON SILVA
ARIEL ANGONESE
EMANUEL SOLIVO
MÁRCIO DAL PIZZOL JÚNIOR**

**CONTROLADOR UART EM PORTAS LÓGICAS:
IMPACTO DE HARDWARE DEDICADO E
INTERRUPÇÕES NO OVERHEAD DE CPU**

ERECHIM – RS

2026

RESUMO

XXXXXXXXXXXXXXXXXXXX

Palavras-chave: XXXXXXXX

ABSTRACT

XXXXXXXXXXXXXXXXXX

Key-words: XXXXXXXXXXXX

LISTA DE FIGURAS

LISTA DE TABELAS

LISTA DE ABREVIATURAS E SIGLAS

CPU – *Central Processing Unit* (Unidade Central de Processamento)

E/S – Entrada e Saída

FPGA – *Field-Programmable Gate Array* (Arranjo de Portas Programável em Campo)

RX – *Receiver* (Receptor)

TX – *Transmitter* (Transmissor)

UART – *Universal Asynchronous Receiver/Transmitter* (Transmissor/Receptor Universal Assíncrono)

SUMÁRIO

1. INTRODUÇÃO	8
2. OBJETIVOS	9
2.1 Objetivos gerais	9
2.2 Objetivos específicos	9
3. REFERENCIAL TEÓRICO	10
4. METODOLOGIA	11
5. RESULTADOS	12
6. CONCLUSÃO	13
REFERÊNCIAS	14

1. INTRODUÇÃO

A evolução dos sistemas computacionais está muito ligada à forma como a unidade central de processamento (CPU) se comunica com os demais componentes de um sistema, especialmente os dispositivos de Entrada e Saída (E/S). Nos primeiros computadores digitais, essa comunicação era realizada de maneira inteiramente controlada por software, em que a própria CPU era responsável por gerenciar cada etapa do envio e recebimento de dados, incluindo tarefas de baixo nível como controle de temporização, verificação de status e montagem ou desmontagem de pacotes de dados bit a bit. Essa abordagem, conhecida como bit-banging quando aplicada à comunicação serial, exige que o processador dedique uma parcela significativa de seu tempo de execução a operações repetitivas e de baixa complexidade lógica, o que compromete sua disponibilidade para realizar outras tarefas de maior relevância computacional.

Com o avanço da arquitetura de computadores, tornou-se evidente a necessidade de designar tarefas específicas e bem definidas a circuitos dedicados, capazes de executá-las de forma autônoma e eficiente, sem exigir intervenção constante da CPU. Esse princípio de especialização de hardware é um dos pilares da computação moderna e está presente em praticamente todos os subsistemas atuais, como controladores de disco, placas de rede, controladores de vídeo e módulos de comunicação serial.

Paralelamente a essa especialização de hardware, também foram desenvolvidos mecanismos mais eficientes de sincronização entre CPU e periféricos, notadamente a substituição gradual da técnica de polling, na qual o processador consulta repetidamente o status de um dispositivo em um laço de espera ativa, pelo uso de interrupções, em que o próprio periférico sinaliza à CPU o momento exato em que uma ação é necessária. Essa mudança de estratégia permite que o processador execute outras tarefas enquanto aguarda a resposta de um periférico, sendo interrompido apenas quando estritamente necessário.

Um exemplo consolidado e amplamente documentado dessa evolução histórica é o protocolo UART (Universal Asynchronous Receiver/Transmitter), utilizado para comunicação serial assíncrona entre dispositivos digitais. Esse protocolo foi historicamente empregado em portas seriais padrão RS-232, presentes na grande maioria dos computadores pessoais até o início dos anos 2000, e permanece amplamente utilizado até os dias atuais em microcontroladores, sensores, módulos de comunicação sem fio, leitores de código de barras e diversos outros dispositivos embarcados, devido à sua simplicidade de implementação,

baixo custo e robustez frente a diferentes cenários de uso. A estrutura de um quadro UART, composta por bit de start, bits de dados, bit de paridade opcional e bit de stop, torna esse protocolo particularmente didático para o estudo de conceitos fundamentais de sistemas digitais, como geração de temporização por meio de contadores, serialização de dados por meio de registradores de deslocamento e verificação de integridade por meio de lógica combinacional.

Nesse contexto, o presente trabalho propõe a construção de um controlador UART híbrido, combinando um módulo transmissor implementado em lógica discreta, utilizando circuitos integrados da família 74xx, com um módulo receptor implementado em uma plataforma de hardware programável (microcontrolador Arduino, baseado no ATmega328P). Essa abordagem serve como base experimental para o estudo. Ela permite analisar a implementação física do protocolo em Sistemas Digitais e investigar, na prática, como as estratégias de comunicação e sincronização afetam o desempenho da CPU na área de Arquitetura de Computadores.

PROBLEMA DE PESQUISA

Embora a existência de controladores dedicados de Entrada e Saída seja um conceito amplamente consolidado na literatura de arquitetura de computadores, seu impacto quantitativo sobre o desempenho de uma CPU real nem sempre é evidenciado de forma prática durante a formação de estudantes da área. Grande parte do conteúdo relacionado a esse tema é apresentada de maneira teórica e conceitual, descrevendo qualitativamente que o uso de hardware dedicado e de interrupções tende a reduzir a carga de trabalho da CPU, sem apresentar dados experimentais que permitam mensurar o tamanho real dessa redução em um sistema físico concreto. Da mesma forma, a comparação entre os modelos de atendimento a periféricos por polling e por interrupção costuma ser discutida em termos de vantagens e desvantagens gerais, sem uma demonstração específica de quanto tempo de processamento ou quantos ciclos de clock são efetivamente economizados ao se adotar um modelo em detrimento do outro.

Essa lacuna entre o conhecimento teórico e a evidência experimental motiva a proposta deste trabalho, que busca investigar de forma prática e mensurável o seguinte problema de pesquisa: qual é o impacto do uso de hardware dedicado, na forma de um controlador UART implementado externamente à CPU, e do uso de interrupções, em

comparação com a técnica de polling, sobre o overhead de processamento de uma unidade central de processamento, medido em ciclos de clock e em tempo de execução? Para responder a essa pergunta, este trabalho propõe a construção de um controlador UART híbrido e a condução de um experimento controlado, no qual quatro cenários distintos de comunicação serial, resultantes da combinação entre transmissão por software (bit-banging) ou por hardware dedicado, e entre atendimento por polling ou por interrupção, são avaliados sob as mesmas condições, permitindo uma comparação direta e quantitativa entre eles.

1.2 Justificativa

A escolha deste tema se justifica, em primeiro lugar, pela relevância acadêmica e técnica do assunto. A relação entre CPU e periféricos, os mecanismos de Entrada e Saída e a distinção entre processamento controlado por software e por hardware dedicado constituem conteúdos centrais nas disciplinas de Arquitetura de Computadores e Sistemas Digitais, mas costumam ser absorvidos de forma abstrata, por meio de diagramas e simulações, sem contato direto com a construção física dos circuitos envolvidos ou com a medição real dos efeitos discutidos em sala de aula. Além disso, o protocolo UART, apesar de sua simplicidade relativa, permanece amplamente utilizado em sistemas embarcados e microcontroladores, o que garante ao trabalho aplicabilidade prática, ao mesmo tempo em que compreender sua construção a partir de contadores, registradores de deslocamento e portas lógicas de paridade fornece uma base sólida para o entendimento de protocolos de comunicação mais complexos.

Além disso, o trabalho se justifica pela oportunidade de integração curricular que proporciona. Por ser desenvolvido de forma concomitante às disciplinas de Sistemas Digitais e Arquitetura de Computadores, o projeto permite aplicar os conteúdos de ambas em um único objeto de estudo, em vez de tratá-los isoladamente: a construção física do módulo transmissor, em lógica discreta, atende aos objetivos formativos de Sistemas Digitais, enquanto a análise experimental do overhead de processamento, medido em ciclos de clock sob diferentes condições controladas, atende aos objetivos de Arquitetura de Computadores. Essa integração reflete, ainda, uma prática comum na engenharia de sistemas computacionais reais, na qual soluções de hardware e software raramente são projetadas de forma isolada, permitindo aos integrantes do grupo validar empiricamente princípios normalmente apresentados apenas de forma teórica.

2. OBJETIVOS

2.1 Objetivos gerais

Este trabalho tem como objetivo geral desenvolver um controlador de comunicação UART (Universal Asynchronous Receiver/Transmitter) híbrido, composto por um módulo transmissor implementado em lógica discreta e um módulo receptor implementado em um microcontrolador (Arduino, baseado no ATmega328P), com a finalidade de analisar e quantificar, de forma experimental, o impacto do uso de hardware dedicado de Entrada e Saída e do mecanismo de interrupções sobre a redução do overhead de processamento e do consumo de ciclos de clock de uma unidade central de processamento. Pretende-se, com isso, evidenciar de maneira prática e mensurável os ganhos de eficiência proporcionados por controladores de comunicação dedicados frente a soluções inteiramente controladas por software, contribuindo para a compreensão de um dos princípios fundamentais da arquitetura de computadores, que é a divisão de responsabilidades entre processamento central e periféricos especializados.

2.2 Objetivos específicos

- Projetar e montar, em lógica discreta com circuitos integrados da família 74xx, o módulo transmissor da UART, responsável pela geração do sinal de baud rate, pela serialização dos dados por meio de registrador de deslocamento e pela geração do bit de paridade.
- Implementar, em firmware executado no microcontrolador, o módulo receptor da UART, responsável pela detecção do bit de start, pela sincronização da amostragem e pela reconstituição do byte recebido.
- Integrar os módulos transmissor e receptor, verificando a integridade dos sinais transmitidos e a correção dos dados recebidos.
- Implementar, no microcontrolador, os quatro cenários experimentais a serem comparados, resultantes da combinação entre transmissão por software (bit-banging) ou por hardware dedicado, e entre atendimento por polling ou por interrupção.
- Quantificar e comparar o overhead de processamento em cada um dos quatro cenários, medido em ciclos de clock e em tempo de execução.

- Analisar os resultados obtidos, discutindo o impacto do uso de hardware dedicado e de interrupções sobre o desempenho da CPU, em relação às abordagens totalmente controladas por software.

3. REFERENCIAL TEÓRICO

3.1 Comunicação Serial E O Protocolo Uart

A comunicação entre dispositivos digitais pode ocorrer de duas formas principais quanto à disposição dos bits transmitidos: paralela, em que múltiplos bits são enviados simultaneamente por diversas linhas físicas, e serial, em que os bits são enviados um a um, sequencialmente, por uma única linha. Embora a comunicação paralela permita, em teoria, taxas de transferência mais altas por ciclo de clock, ela exige um número maior de condutores físicos, o que aumenta o custo, a complexidade e a suscetibilidade a interferências eletromagnéticas à medida que a distância entre os dispositivos aumenta. Por esse motivo, a comunicação serial se consolidou como a abordagem preferida para a maioria das aplicações de comunicação entre periféricos e unidades de processamento, especialmente em sistemas embarcados.

Dentro desse contexto, o protocolo UART se destaca como um dos métodos mais utilizados de comunicação serial assíncrona. Diferentemente da comunicação síncrona, que depende de um sinal de clock compartilhado entre transmissor e receptor, a comunicação assíncrona não exige uma linha de clock dedicada, sendo a temporização de cada bit determinada por uma taxa de transmissão previamente acordada entre os dois dispositivos, conhecida como baud rate. Segundo Longadge, Mahindra e Dahake (2015), a UART é composta essencialmente por três componentes principais: o transmissor, o receptor e o gerador de baud rate, sendo responsável por converter dados paralelos, provenientes do barramento interno de um sistema computacional, em uma sequência serial de bits, e vice-versa.

A estrutura de um quadro (frame) de dados UART segue um formato bem definido. A transmissão de cada byte é iniciada por um bit de start, que sinaliza ao receptor o início de um novo quadro e permite a sincronização momentânea entre os dois dispositivos. Em seguida, são transmitidos os bits de dados propriamente ditos, geralmente sete ou oito bits, começando pelo bit menos significativo. Opcionalmente, pode ser incluído um bit de paridade, utilizado para verificação básica de integridade da informação transmitida. Por fim, um ou mais bits de stop marcam o final do quadro, sinalizando ao receptor que a linha pode

retornar ao seu estado de repouso até o início da próxima transmissão. Ainda de acordo com Longadge, Mahindra e Dahake (2015), esse formato de quadro faz com que a transmissão de um byte de dados efetivamente exija a transmissão de dez bits quando consideradas as sobrecargas de start, paridade e stop, o que evidencia a diferença entre a taxa de baud (número de mudanças de estado do sinal por segundo) e a taxa efetiva de transferência de dados.

O baud rate de uma transmissão UART é determinado por um gerador de temporização que, tipicamente, divide a frequência de um sinal de clock de referência até atingir a frequência correspondente à taxa desejada, permitindo que tanto o transmissor quanto o receptor amostram cada bit no instante correto de sua duração. Essa característica torna o protocolo UART especialmente adequado ao estudo de conceitos fundamentais de sistemas digitais, uma vez que sua implementação física depende diretamente de blocos lógicos sequenciais bem estabelecidos, como contadores para geração de temporização e registradores de deslocamento para a conversão entre os formatos paralelo e serial dos dados, que são discutidos em detalhe na seção seguinte.

3.2 Lógica Sequencial: Contadores E Registradores De Deslocamento

Os circuitos digitais podem ser classificados, de forma geral, em combinacionais e sequenciais. Enquanto os circuitos combinacionais têm suas saídas determinadas exclusivamente pelos valores presentes em suas entradas, os circuitos sequenciais possuem a capacidade de armazenar informação ao longo do tempo, de modo que suas saídas dependem não apenas das entradas atuais, mas também do estado interno previamente armazenado. Essa capacidade de memorização é obtida por meio de elementos básicos de armazenamento, os flip-flops, que constituem a unidade fundamental a partir da qual são construídos praticamente todos os circuitos sequenciais utilizados em sistemas digitais, incluindo registradores e contadores.

Um registrador de deslocamento (shift register) é um circuito sequencial formado por uma cadeia de flip-flops conectados em cascata, de tal forma que a saída de cada flip-flop é conectada à entrada do flip-flop seguinte. A cada pulso de clock aplicado ao circuito, o conteúdo armazenado em cada flip-flop é transferido para o flip-flop adjacente, produzindo um deslocamento do dado ao longo da cadeia. De acordo com De Castro (2009), essa estrutura permite duas configurações principais de operação: o registrador com entrada serial

e saída paralela, no qual os bits são inseridos um a um pela entrada serial e podem ser lidos simultaneamente em todas as saídas após o preenchimento completo do registrador, e o registrador com entrada paralela e saída serial, no qual um dado é carregado simultaneamente em todos os flip-flops e, em seguida, deslocado para fora do circuito um bit por vez através da saída serial.

É justamente essa segunda configuração que fundamenta a operação do módulo transmissor de um controlador UART, uma vez que ela realiza exatamente a conversão de um byte de dados, disponível em paralelo no barramento interno de um sistema, para o formato serial exigido pela linha de transmissão. De maneira complementar, o módulo receptor de uma UART utiliza a configuração inversa, reconstituindo em paralelo o byte recebido bit a bit pela entrada serial. Não por acaso, De Castro (2009) destaca explicitamente que a comunicação entre computadores através de suas portas seriais tem, na sua essência, a utilização de registradores de deslocamento para realizar essa conversão entre os formatos paralelo e serial dos dados.

Já um contador é um circuito sequencial cujo objetivo é gerar uma sequência numérica predeterminada, tipicamente incrementada a cada pulso de clock recebido. Os contadores podem ser classificados em duas grandes categorias, de acordo com a forma como o sinal de clock é distribuído entre seus flip-flops. Nos contadores assíncronos, também chamados de contadores por pulsação (ripple counters), apenas o primeiro flip-flop do circuito recebe diretamente o sinal de clock externo. Neles os demais flip-flops são disparados pela transição de saída do flip-flop imediatamente anterior na cadeia, o que gera um efeito de propagação sucessiva ao longo do circuito. Já nos contadores síncronos, todos os flip-flops compartilham o mesmo sinal de clock, sendo habilitados a mudar de estado simultaneamente, o que elimina os atrasos característicos dos contadores assíncronos e evita a ocorrência de sinais falsos e rápidos (glitches) nas saídas do circuito, conforme discutido por De Castro (2009).

No contexto deste trabalho, um contador é utilizado como elemento central do gerador de baud rate do módulo transmissor da UART, sendo responsável por dividir a frequência de um sinal de clock de referência até atingir a frequência correspondente à taxa de transmissão desejada. Esse sinal de temporização, por sua vez, é utilizado para controlar a cadência com que o registrador de deslocamento libera cada bit do quadro de dados na linha

de transmissão, evidenciando a relação direta e complementar entre esses dois blocos fundamentais de lógica sequencial na construção física de um controlador UART discreto.

3.3 Entrega E Saída Em Arquitetura De Computadores: Polling E Interrupção

Um dos aspectos centrais no projeto de um sistema computacional é a forma como a unidade central de processamento (CPU) se comunica com seus dispositivos periféricos denominados subsistema de Entrada e Saída (E/S). Diferentemente da memória principal, cujo tempo de acesso é relativamente previsível e rápido, os dispositivos de E/S apresentam grande variação em suas velocidades de operação e, em muitos casos, exigem que a CPU aguarde a disponibilidade de um dado ou a conclusão de uma operação antes de prosseguir. Essa característica torna a estratégia de sincronização entre CPU e periférico um fator determinante para o desempenho geral do sistema, dando origem, historicamente, a dois modelos principais de atendimento a periféricos: o polling e a interrupção.

No modelo de polling, também chamado de E/S controlada por programa ou espera ativa, a CPU consulta repetidamente um registrador de status do periférico, em um laço de verificação contínuo, até que este indique que uma condição específica foi atendida, como a disponibilidade de um novo dado recebido ou a conclusão de uma operação de transmissão. Embora essa abordagem seja conceitualmente simples de implementar, tanto em termos de hardware quanto de software, ela apresenta uma limitação significativa: durante todo o período de espera, a CPU permanece inteiramente dedicada à tarefa de verificação, sem capacidade de executar qualquer outra instrução útil, o que resulta em desperdício de ciclos de processamento proporcional ao tempo de espera pelo periférico.

Como alternativa a essa limitação, foi desenvolvido o modelo de atendimento por interrupção, onde a CPU não precisa consultar ativamente o status do periférico. Em vez disso, o próprio periférico é responsável por sinalizar à CPU, por meio de uma linha de interrupção dedicada, o momento exato em que uma condição relevante ocorre. Ao receber esse sinal, a CPU suspende temporariamente a execução do programa atual, desviando o fluxo de execução para uma rotina de tratamento de interrupção (interrupt handler), responsável por atender à necessidade específica do periférico. Ao término dessa rotina, a CPU retoma a execução do programa original a partir do ponto em que havia sido interrompido. Esse mecanismo permite que a CPU permaneça livre para executar outras tarefas durante o intervalo em que o periférico ainda não está pronto, sendo desviada de sua

execução apenas no momento estritamente necessário, o que reduz significativamente o overhead de processamento em comparação à espera ativa característica do polling.

Essa diferença entre os dois modelos, embora amplamente discutida na literatura de arquitetura de computadores em termos qualitativos, também pode ser evidenciada de forma quantitativa. O estudo conduzido por Koh et al. (2019), voltado à análise de dispositivos de armazenamento de baixíssima latência, mediu de forma experimental o percentual de utilização de CPU exigido por diferentes estratégias de atendimento a operações de Entrada e Saída, demonstrando que o modelo de polling pode consumir uma parcela significativamente maior dos ciclos de processamento disponíveis em comparação ao modelo baseado em interrupções, especialmente à medida que a frequência de eventos de E/S aumenta. Esses resultados reforçam a relevância prática da distinção entre os dois modelos e fundamentam a proposta experimental deste trabalho, que busca reproduzir, em menor escala e utilizando um controlador UART como periférico de referência, uma comparação quantitativa semelhante entre polling e interrupção. Além disso, ela também é complementada pela comparação adicional entre comunicação serial realizada inteiramente por software (bit-banging) e comunicação realizada por meio de hardware dedicado

4. METODOLOGIA

5. RESULTADOS

6. CONCLUSÃO

REFERÊNCIAS

DE CASTRO, F. C. C. Eletrônica Digital: Capítulo VIII - Registradores de Deslocamento e Contadores. Porto Alegre: Pontifícia Universidade Católica do Rio Grande do Sul, Faculdade de Engenharia Elétrica, 2009. Disponível em: http://www.fccdecastro.com.br/pdf/ED_C8.pdf. Acesso em: 23 ago. 2026.

KOH, S.; JANG, J.; LEE, C.; KWON, M.; ZHANG, J.; JUNG, M. Faster than Flash: An In-Depth Study of System Challenges for Emerging Ultra-Low Latency SSDs. arXiv preprint arXiv:1912.06998, 2019. Disponível em: <https://arxiv.org/pdf/1912.06998>. Acesso em: 24 ago. 2026.

LONGADGE, R.; MAHINDRA, M.; DAHAKE, V. **Design and Implementation of UART**. International Journal of Advanced Research in Computer and Communication Engineering, v. 4, n. 5, p. 315-317, maio 2015. Disponível em: <https://www.ijarcce.com/upload/2015/may-15/IJARCCE%2070.pdf>. Acesso em: 24 ago. 2026.

**URI**UNIVERSIDADE REGIONAL INTEGRADA
DO ALTO URUGUAI E DAS MISSÕES

8. MATRIZ BIBLIOGRÁFICA

Fonte	Tipo	Tema Abordado	Contribuição para o projeto
DE CASTRO, F. C. C. (2009)	Material didático (PUCRS)	Registradores de deslocamento e contadores: lógica sequencial, contadores síncronos e assíncronos	Fundamenta os dois blocos de hardware discreto centrais do projeto (gerador de baud rate por contador e serialização por shift register), com base teórica direta para o projeto do circuito em 74xx, usada na Seção 3.2
KOH, S. et al. (2019)	Artigo científico (arXiv)	Comparação experimental entre polling e interrupção: overhead de CPU	Fornece precedente metodológico e evidência empírica de que polling gera maior consumo de ciclos de CPU que interrupção, servindo

**URI**UNIVERSIDADE REGIONAL INTEGRADA
DO ALTO URUGUAI E DAS MISSÕES

		medido quantitativamente	de referência direta para o desenho do experimento 2x2 e para a discussão dos resultados esperados, usada na Seção 3.3
LONGADGE, R.; MAHINDRA, M.; DAHAKKE, V. (2015)	Artigo científico (IJARCCE)	Protocolo UART: estrutura do frame, bits de start/stop/paridade, gerador de baud rate	Fundamenta teoricamente o protocolo que será implementado fisicamente no módulo TX, especialmente a lógica de formação do quadro de dados e a relação entre baud rate e taxa efetiva de transferência, usada na Seção 3.1